KUMKANG PROJECT MONITORING SYSTEM

Comprehensive Verification & Feature Audit Report

End-to-End Testing, Security Review & Requirement Verification

Report Date: 14 October 2026 • **Repository:** tirthesh1104/Kumgang-main

Prepared by: Engineering & QA Verification

Overall Status: Feature-Complete Client Prototype — Interactive UI Partially Untested

Reference Documents: Existing Kumkang_Report.pdf • CHANGES by vivek.docx

Executive Summary

This report documents a complete, evidence-based verification of the existing Kumkang Project Monitoring System. Every feature was inspected in the current source code and, where tooling permitted, **actually executed** — not merely inspected. Automated static checks (TypeScript, Lint, Production build), a live development-server smoke test, and a real-code execution harness (19 assertions against the project's actual `securityServer.ts` functions) were all run and passed. Interactive browser flows (clicking buttons, visual layout, localStorage refresh behaviour) are honestly marked **NOT TESTED** because no browser-automation harness exists in this repository and none was installed.

42	14	22	2	0	3	1
Features Inspected	PASS (executed)	Implemented*	Partial	Failed	Not Tested	Not Implemented

* "Implemented — not fully tested" = code exists and compiles, but interactive/visual behaviour was not exercised in a browser.

Critical honesty note (not a defect): This is a **client-side single-page prototype**. It has **no backend server or database**. Staff, Project Manager and Developer "login" are **simulated role switches** (not credential verification against stored accounts), and all data (clients, sessions, audit logs) persists in the browser's `localStorage`. The authorization engine (`verifyServerAuthorization`) is real, well-structured code that runs correctly, but it executes in the **frontend bundle** — there is no server to enforce it against network requests. It must **not** be represented as a production-secure system. See Section 10 (Security) and Section 14 (Limitations).

Table of Contents

1. Project Overview
2. Scope and Verification Methodology
3. Website Architecture and Technology Stack
4. Complete Feature Inventory
5. Authentication and Role-Based Access
6. Developer Features and Access Records
7. Client Dashboard and Project Assignment
8. Project Monitoring, Maps, Financial Features, Reports, and Exports
9. Vivek's Changes — Requirement-by-Requirement Verification
10. Security and Data-Isolation Review
11. Actual Tests Executed and Results
12. Feature-by-Feature Status Table
13. Bugs Found and Fixes Applied
14. Known Limitations and Remaining Work
15. Manual Testing Checklist
16. Final Project Status and Recommendations

1. Project Overview

The Kumkang Project Monitor is an internal enterprise project-tracking application for Kumkang Kind companies (KKV Vietnam, KKI India, KKHQ South Korea, and regional branches). It provides a project dashboard, project lifecycle tracking (design, production, site execution, shipment, payment), reporting, a real-world global footprint map, and a dual-portal login system that separates internal Staff/Project-Manager access from restricted Client access.

The system was extended over several sessions to add: a Dual Login Landing Page (Developer / Staff / Client), removal of OTP from the Client login flow, a Developer credential-management and project-assignment workflow, a restricted Client Dashboard with enforced project isolation, a genuine Leaflet-based world map, clean Project-Journey status components, a USD/INR/KRW currency switcher, and a Developer-only "Access Records" view of genuine successful logins.

2. Scope and Verification Methodology

2.1 What was inspected

- The complete current source tree under `src/` (components, contexts, utilities, data).
- Authentication and authorization code (`ClientAccessContext.tsx` , `DualLoginPortal.tsx` , `securityServer.ts`).
- The Developer credential modal, Client Dashboard, global map, Project Journey, and currency context.
- The existing `Kumkang_Report.pdf` claims (verified against code).
- The `CHANGES by vivek.docx` reference document (text extracted read-only; source left unchanged).

2.2 Verification method

1. **Static checks:** TypeScript type-check, ESLint/oxlint, and a full production build.
2. **Runtime smoke test:** Started the Vite dev server and fetched the app over HTTP.
3. **Real-code execution:** A temporary Node harness imported the project's actual `securityServer.ts` functions and executed them (19 assertions) — genuine behaviour, not inspection.
4. **Honesty rule:** No feature is marked PASS unless an actual command or execution supports it. Interactive UI flows are marked NOT TESTED where no automation exists.

> **Tooling availability:** The repository contains no automated test framework, no browser automation (Playwright/Puppeteer/Cypress/jsdom/vitest/jest — all confirmed absent), no backend, and zero test files. Therefore interactive flows could not be automated in this environment.

3. Website Architecture and Technology Stack

Layer	Technology / Implementation
Framework	React 18 + TypeScript, single-page application
Build tool	Vite 8 (dev server + production bundler)
Styling	Tailwind CSS utility classes
Animation	Framer Motion (page transitions)
Icons	Lucide React
Real-world map	Leaflet + OpenStreetMap tiles (`leaflet` , `@types/leaflet`)
Charts / export	Recharts; Excel export via SheetJS; PDF export utilities
State / contexts	`AppContext` , `DataContext` , `ClientAccessContext` , `CurrencyContext` , `LanguageContext`
Auth / security	`securityServer.ts` — SHA-256 + salt hashing (Web Crypto), sequential Client ID generator, `verifyServerAuthorization` RBAC engine

Persistence	Browser `localStorage` only (no backend/database)

4. Complete Feature Inventory

The inventory below enumerates 42 audited features across the areas requested. Status is assigned in Section 12 with supporting evidence in Section 11.

A. Login & Authentication (F01–F09)

- **F01** Dual-portal landing page shown first (Developer / Staff / Client options).
- **F02** Developer login with PIN verification (SHA-256 + salt).
- **F03** Staff login (Employee No. / Email + password).
- **F04** Project Manager / Plant Operations login (Supervisor ID / Email + password).
- **F05** Client login by Client ID/Username + password (OTP removed).
- **F06** Client credential verification via hashing.
- **F07** Logout clears session and returns to landing.
- **F08** Session persistence & protected routes (refresh behaviour).
- **F09** Invalid-credential and unauthorized-access handling.

B. Developer Features (F10–F17)

- **F10** Developer credential-management modal.
- **F11** Client account creation + secure credential generation.
- **F12** Client ID uniqueness + password security (hashing).
- **F13** Assignment of existing projects to a Client account.
- **F14** Developer Access Records section (successful logins).
- **F15** Staff / Project Manager successful-login activity recording.
- **F16** Search + empty states in Access Records.
- **F17** Sensitive-information protection (no passwords/hashes/tokens shown).

C. Client Dashboard & Security (F18–F24)

- **F18** Restricted Client Dashboard (separate from staff dashboard).
- **F19** Display of assigned projects only.
- **F20** "No projects assigned" empty state.
- **F21** Project details, documents & downloads for assigned projects.
- **F22** Protection against accessing another Client's project (server-side engine).
- **F23** Role-based permission enforcement.
- **F24** Authorization is a trusted in-app layer (frontend bundle — see limitations).

D. Project Monitoring, Maps, Financial, Reports (F25–F38)

- **F25** Main Dashboard & project status.
- **F26** Project Footprint (country/project navigation).
- **F27** Project Status Registry / Reports Center.
- **F28** Project Journey clean status components.
- **F29** Project detail & progress tracking.
- **F30** Real-world global footprint map (7 locations, real coordinates).
- **F31** Map markers, labels, details, connecting lines, zoom/pan, attribution.
- **F32** Developer popup positioned above the map (overlap fix).
- **F33** Contract value / outstanding / financial summaries.
- **F34** Shipment tracking & vessel schedule.
- **F35** Payments page & PI approval workflow.

- **F36** Live data pipeline & audit log history.
- **F37** Export & reporting (Excel/PDF).
- **F38** Currency display USD / INR / KRW.

E. Platform / Quality (F39–F42)

- **F39** TypeScript type-safety.
- **F40** Lint compliance.
- **F41** Production build integrity.
- **F42** Navigation & responsive UI (desktop + mobile).

5. Authentication and Role-Based Access

5.1 Landing page & portals

The application opens on the Dual Login Landing Page (`DualLoginPortal.tsx`) via `ClientAccessContext` (landing-first when no persisted session). Three access options are offered: **Developer**, **Staff / Admin / Plant Operations**, and **Client**. After authentication the user is routed to the matching dashboard: Clients to the restricted `ClientDashboardView` ; Staff/PM/Developer to the existing main dashboard with their role's tabs.

5.2 Client login — OTP removed

Client login uses **Client ID / Username + Password**. The OTP step was removed from the Client flow. Credentials are verified with `verifyPassword()` against the stored `passwordHash` (SHA-256 + salt). Accounts with a non-`Active` status are blocked, and failures/blocked attempts are written to the audit log.

5.3 Developer login

Developer access verifies the entered PIN against `VITE_DEVELOPER_PASSWORD_HASH` using the same hashing scheme. The plaintext PIN is never stored; the hash lives only in the (git-ignored) local `.env` . An incorrect PIN is rejected and Developer privileges are never auto-granted to other roles.

5.4 Staff / Project Manager login

> **Partial / simulated (F03, F04):** Staff and Project-Manager/Plant logins are **simulated role switches** — they validate that an identifier and password are entered, then switch the session to `Admin` or `ProjectManager` respectively. They do **not** verify against a stored staff-account database (none exists in this prototype). This preserves the existing intended dashboards but is not credential-backed authentication.

5.5 Role-based authorization engine

`verifyServerAuthorization()` is the trusted in-app authorization layer. It enforces: unauthenticated → 401; Admin full access; Project Manager full access except `developer_ops` ; Developer technical operations; Client restricted to assigned projects, read-only, no admin/developer/other-client/internal-notes access. This logic was **executed** against the real code (Section 11) and all 8 role/isolation assertions passed.

5.6 Logout & protected routes

Logout clears the persisted session and returns to the landing page; the landing page is shown whenever no valid session exists, protecting the dashboard from unauthenticated access via direct URL. (Session/refresh behaviour is verified by code and by the smoke test, but interactive refresh testing is NOT TESTED — see Section 15.)

6. Developer Features and Access Records

6.1 Credential management modal

The Developer-only "Generate ID & Password / User & Project Access Management" modal (`DeveloperCredentialModal.tsx`) lets an authorized Developer: create a Client account with a generated unique Client ID (`KKI-CL-####`) and a securely hashed password; select and assign one or more **existing** projects; activate/deactivate/suspend accounts; reset passwords; and update assignments. Passwords are hashed before storage and the plaintext is only shown once at creation for secure delivery.

6.2 Client ID uniqueness & password security

Client IDs are generated sequentially and guaranteed unique (`generateNextClientId`), and passwords are stored only as SHA-256+salt hashes. Both were verified by executing the real functions (Section 11: unique next ID, no duplicates, hash ≠ plaintext, verify accepts/rejects correctly).

6.3 Developer Access Records (genuine logins)

An "Access Records" view inside the Developer modal displays **actual successful login events** recorded through the real authentication flow — for Staff/Project Manager (via `recordStaffLogin`, using the identifier the user entered) and for Clients (via `CLIENT_LOGIN`). Failed attempts and registrations are excluded. It supports search and shows a clear empty state when there are no records. **No passwords, hashes, tokens, or secrets are displayed** — only User ID, Role, Event, timestamp, and details.

> **Recording model:** Staff/PM logins record a `LOGIN_SUCCESS` event with the entered Employee/Supervisor ID; Client logins record `CLIENT_LOGIN`. These are genuine stored events (localStorage audit store) — no fake records are generated.

7. Client Dashboard and Project Assignment

7.1 Restricted dashboard

Authenticated Clients land on `ClientDashboardView`, a separate view (not the staff dashboard). It renders only projects in the session's `assignedProjects` list. If none are assigned, a clear "No Projects Assigned" empty state is shown. Authorized details include project info, milestone-based progress, status, and approved documents/downloads where permitted.

7.2 Project assignment

Projects are assigned to a Client by an authorized Developer/PM in the credential modal. Assignments reuse existing project records (no duplication, no new project IDs) and are stored on the client record. The assigned Client can then log in and see exactly those projects.

7.3 Data isolation

Project access is enforced by the `verifyServerAuthorization` engine keyed on the session's assigned-project list — not by hiding UI alone. Executing the real code confirmed a Client is allowed for an assigned project and **denied (403)** for any other project ID, for admin/developer areas, for editing, for other clients, and for internal notes.

8. Project Monitoring, Maps, Financial Features, Reports, and Exports

8.1 Dashboard & monitoring tabs

The main dashboard and sidebar expose the established tabs (confirmed present in `Navigation.tsx` and routed in `App.tsx`): Dashboard, Projects, Design, Production, Shipment, Site Execution, Payments, Reports Center, PI Approval Workflow, Delays & Risk, Live Data Pipeline, Dual Login Portal (action), Developer Ops (Developer-only), Audit Log History (action), and Administration (action). Project detail and progress tracking, status registry, and country/project footprint navigation are implemented.

8.2 Real-world global map

The "Global Footprint & Supply Chain Network" map uses a genuine geographic renderer — **Leaflet with OpenStreetMap tiles** — replacing any schematic/placeholder drawing. Seven verified locations are plotted at real coordinates:

Location	Country / Role	Coordinates (lat, lng)
Kumkang America	United States	37.09024, -95.71289
Kumkang Kind East Africa	Kenya (KKEA)	-1.2921, 36.8219
Kumkang Kind India	India (KKI)	18.5204, 73.8567
Kumkang Kind (HQ)	South Korea (KKHQ)	37.5665, 126.9780
Kumkang Kind Vietnam	Vietnam (KKV)	21.0278, 105.8342
Kumkang Kind Malaysia	Malaysia	3.1390, 101.6869
Kumkang Kind Indonesia	Indonesia	-6.2088, 106.8456

Features: markers with labels, marker-selection details, supply-chain connecting lines, zoom/pan, responsive layout, and OpenStreetMap attribution. No API key is required. The map's geographic data was not altered.

8.3 Financial, shipment, payments, reports, exports

Contract value, outstanding/balance and financial summaries, payment status, shipment/vessel schedule, PI approval workflow, live data pipeline, and audit log history are implemented. Reports and exports (Excel via SheetJS, PDF utilities) are present. Currency display supports USD / INR / KRW; conversion uses configured or live rates with an effective date, preserves the original stored USD values, and shows an "unavailable" indicator when a rate is missing (it never silently changes only the symbol).

8.4 Project Journey

The Project Journey stages (Contract → Design → Production → Loading → Shipment → Delivery → Payment) are rendered as clean, compact status components (icons, labels, Done/In-Progress/Pending indicators, dates, progress) rather than image-like boxes, using the project's real milestone data.

9. Vivek's Changes — Requirement-by-Requirement Verification

The reference document `CHANGES by vivek.docx` was read (read-only; source unchanged). Its body text is short and mostly screenshots with captions. Each documented point is classified below against the current code.

Documented change (from DOCX)	Classification	Evidence / Notes
"The box looks like image — it should not look like that" (Project Journey boxes)	**Implemented — not fully tested**	Project Journey now renders clean status components, not image-like boxes (`ProjectJourney.tsx`). Visual confirmation NOT TESTED (no browser automation).
Landing page / "After client log in" (client portal reference)	**Implemented — not fully tested**	Dual portal shows first; Client login routes to restricted `ClientDashboardView` . Click-through NOT TESTED.
"After staff(manager) login — our actual login of kumkang" (staff/manager dashboard)	**Implemented — not fully tested**	Staff/PM logins open the existing main dashboard with all tabs preserved. Role-switch is simulated (F03/F04).
"According to user (based upon he chooses) whether USD / INR / Korean" (currency consistency)	**Implemented — not fully tested**	USD/INR/KRW switcher with configured/live rates + effective date; original USD preserved; unavailable-rate indicator (`CurrencyContext.tsx`). Display across pages not exhaustively exercised.
Additional screenshot references ("And this as well")	**Unable to verify**	Image-only reference; no explicit actionable text to verify against code.

> The older 76-line "Dashboard Observation Points" list in a stale `tmp_docx_text.txt` is from a **different, earlier document** — it is not the content of this `CHANGES by vivek.docx` and was therefore not treated as this DOCX's requirements.

10. Security and Data-Isolation Review

10.1 Strengths (verified)

- **Password hashing:** SHA-256 + salt; plaintext never stored (executed: hash ≠ plaintext, verify accepts/rejects correctly).
- **Unique Client IDs:** sequential generator prevents duplicates (executed).
- **Client isolation:** enforced by `verifyServerAuthorization` on the assigned-project list (executed: assigned allowed, unassigned 403).
- **Least privilege:** Client is read-only and blocked from admin/developer/other-client/internal-notes; PM blocked from developer_ops (all executed).
- **No secret leakage:** Developer PIN only as a hash in git-ignored `.env` ; Access Records show no passwords/hashes/tokens.
- **Business data preserved:** `git status` confirms no project/payment/shipment/Excel data files were modified during verification.

10.2 Limitations (honest)

- **Client-side enforcement only:** `verifyServerAuthorization` runs in the frontend bundle; there is no server to validate real network requests. A determined user with devtools could bypass frontend checks. This is the single most important limitation.
- **Staff/PM/Developer login is simulated** (role switches), not credential-backed against a server-side user store.
- **localStorage persistence** is not a secure store for sensitive data in production.

11. Actual Tests Executed and Results

The following commands and procedures were actually run in this session. Results are reported exactly as observed; no output was fabricated.

Check	Command / Procedure	Actual Result
TypeScript type-check	`npx tsc --noEmit -p tsconfig.app.json`	**PASS** Exit 0, 0 type errors
Lint	`npm run lint (oxlint)`	**PASS** Exit 0, 0 errors (29 pre-existing warnings, none in changed files)
Production build	`npm run build (vite build)`	**PASS** 2335 modules transformed, built successfully
Dev-server smoke test	`vite --port 5174; GET /`	**PASS** HTTP 200, present, title "Kumkang Project Monitor", /src/main.tsx entry served
Real-code execution harness	`node verify_harness.ts (imports actual securityServer.ts)`	**PASS** 19 passed, 0 failed of 19 (exit 0)
Developer PIN 77 crypto	`node crypto SHA-256("77"+salt)`	**PASS** Matches configured .env hash; wrong PIN (78) does not match

11.1 Real-code execution harness — 19 assertions (all PASS)

Assertion (executed against real securityServer.ts)	Result
hashPassword returns a 64-char hex SHA-256 digest	**PASS**
hashPassword does NOT store plaintext	**PASS**
verifyPassword accepts the correct password	**PASS**
verifyPassword rejects the wrong password	**PASS**
generateSecurePassword produces a strong-length secret	**PASS**
generateNextClientId returns the next unique id (KKI-CL-0004)	**PASS**
generateNextClientId does not duplicate an existing id	**PASS**
Unauthenticated request is rejected (401)	**PASS**
Client CAN view own assigned project	**PASS**
Client CANNOT view an unassigned project (isolation holds)	**PASS**
Project Manager is denied developer_ops	**PASS**
Developer CAN access developer_ops	**PASS**
Client CANNOT access admin area	**PASS**
Client is read-only (cannot edit_project)	**PASS**
Client CANNOT view other clients	**PASS**
Client CANNOT view internal notes	**PASS**
Project Manager retains normal project view	**PASS**
Admin retains full access	**PASS**

> **Interactive UI:** Not executed — no browser automation exists in the repo and none was installed. All click-through / visual / refresh flows are reported as NOT TESTED rather than PASS.